Assignment 2 — Synchronized Typed Buffer

Topic: Thread synchronization — predicate-based waiting and correct synchronization (a variation of the bounded producer/consumer problem) **Due:** 11:59 PM, November 01, 2026 (Always check Canvas on updated due dates)

---

1. Learning objectives

After completing this assignment you should be able to:

1. Protect shared mutable state with a mutex so that concurrent operations are **atomic** and free of data races.
2. Make a thread **block on a predicate** (a condition about the shared state) using condition variables or semaphores, instead of busy-waiting.
3. **Block and resume correctly**: when a thread cannot proceed it blocks (without busy-waiting), and when its condition holds it resumes — with no lost wakeups and no thread left blocked. Whichever primitive you choose, structure the waiting so a thread resumes only when it can actually proceed.
4. Reason about **liveness and termination**: argue why your design cannot deadlock, livelock, or leave a thread blocked at the end.
5. Make per-thread output **reproducible** even though the interleaving is not.

---

2. Overview

Implement a bounded buffer (capacity 50) shared by **two producers** and **two consumers**. Items are integers, each tagged by parity (odd or even). The twist that makes this a synchronization problem rather than a queueing exercise: **each consumer can only take items of its own type**, so it must block until an item of that type is present.

You must use POSIX threads (pthreads) in C or C++. All four producers and consumers are pthreads created with `pthread_create` and joined with `pthread_join`; synchronization uses the pthreads/POSIX primitives listed in §5. Solutions built on any other threading model or language will not be accepted.

This gives you **three distinct wait conditions** over one shared buffer:

- a consumer waiting for an **odd** item to exist,
- a consumer waiting for an **even** item to exist,
- a producer waiting for **free space**.

Keeping these three conditions distinct — so that a thread blocked on one of them resumes only when *that* condition holds — is the point of the assignment. A single "an item exists" condition cannot tell a consumer waiting for an odd item apart from one waiting for an even item; working out how to separate the three conditions (with whatever primitives you choose) is the core of the work.

The container itself is unimportant. Use any structure you like (array, ring buffer, `std::deque`, `std::vector`) as long as **all locking is your own**. Ordering within the buffer does not matter; you may remove any item of the requested type. Do **not** spend effort on a fancy data structure — spend it on the synchronization.

---

3. The shared buffer

- Holds up to **50 items** at once; each item is an integer `1 ≤ n ≤ 100`, tagged odd or even.
- Order is irrelevant. To remove an item of a given parity you may take any such item (e.g., a linear scan of ≤ 50 elements, or maintain per-parity counts — your choice).
- Track (or be able to compute) three quantities at any time: `size`, `odd_count`, `even_count`. These are exactly the quantities the wait predicates depend on.

---

4. Threads

Producers (2 threads)

- `producer_odd`: repeatedly generates a random **odd** integer in [1, 100] and inserts it.
- `producer_even`: repeatedly generates a random **even** integer in [1, 100] and inserts it.
- Each producer produces **exactly 100 items**, then terminates (§7).
- If the buffer is **full** (`size == 50`), a producer must **block** until space is available — it may not spin.

Consumers (2 threads)

- `consumer_odd`: removes one **odd** item and records the action; if no odd item is present, it must **block** until one is.
- `consumer_even`: removes one **even** item and records the action; if no even item is present, it must **block** until one is.
- A consumer blocks on the presence of *its own type*, not on the buffer being non-empty. A buffer full of even items must not wake or unblock `consumer_odd`.

Because each consumer waits only on its **own** item type, neither is blocked merely because the other is the one with an item to take — both make forward progress over the run. This is about the wait conditions, not simultaneous access: the consumers still take the buffer lock

one at a time for each removal, so only one thread ever modifies the buffer at a given instant.

---

5. Synchronization requirements

- Use **POSIX threads** (`pthread_create`) in C or C++.
- Allowed primitives: `pthread_mutex_t` , `pthread_cond_t` , POSIX semaphores (`sem_t`). Choose your own combination.
- **No busy-waiting.** A thread that cannot proceed must block on a condition variable or semaphore rather than spinning. (If you use condition variables, re-checking the predicate in a `while` loop around the wait is required to handle spurious and stale wakeups; that loop is not busy-waiting.)
- **Mutual exclusion.** Every access to the buffer and its counts happens while holding the buffer lock. Inspecting the state and acting on it (e.g., "is there an odd item? if so remove one") must be a **single critical section** so the state cannot change between the check and the action.
- **No lost wakeups.** Whenever an operation changes the shared state, any thread whose condition has now become true must be able to resume. In particular, inserting an item may make an odd- or even-waiting consumer runnable, and removing an item makes a space-waiting producer runnable. No eligible thread may be left blocked. How you guarantee this depends on your primitives: a counting semaphore remembers a `post` that arrives before anyone is waiting, whereas a condition variable does not — so with condition variables a single mis-targeted `signal` on a shared condition can leave the wrong consumer asleep forever. Either approach is fine as long as no wakeup is lost.
- **Targeted blocking (graded, §13).** Beyond mere correctness, a thread should resume only when it can actually proceed, rather than repeatedly waking to find it still cannot. Blocking each waiter on the specific condition it needs achieves this — for example separate semaphores (or separate condition variables) for odd-available, even-available, and space-available — whereas funneling everything through one shared condition and rousing all waiters on every change is correct but wasteful.
- **No deadlock, livelock, or indefinite blocking** under the test inputs, and no thread left blocked after all work is done (§7).

---

6. Randomness and reproducibility

- Accept an optional seed: `-s <seed>` . **Default seed is 42.**
- Each producer's **sequence of generated values must be reproducible** for a given seed. Do **not** call `rand()` / `random()` from multiple threads — they are not thread-safe and their output would depend on the interleaving. Give each producer its **own** pseudo-random number generator (PRNG) state (e.g., `rand_r()` with a per-thread

seed deterministically derived from the base seed, such as `seed` and `seed + 1`). A PRNG produces a repeatable sequence of values from its seed, so a per-thread seed gives each producer its own reproducible stream.
- The interleaving/trace is **not** reproducible, and that is expected. Reproducible *values* plus a nondeterministic *schedule* is the point.

---

7. Termination

- Each **producer** exits after producing its **100 items**.
- Each **consumer** exits after removing its **100 items**. Treat this fixed count as the termination condition. It is exact because of the type invariant: only `consumer_odd` removes odd items and only `consumer_even` removes even items, so each consumer consumes exactly the 100 items its matching producer created. **Do not** terminate a consumer on "buffer empty and producers finished" — that can hang; use the count.
- After the last item is consumed, **all four threads must terminate cleanly** with **no thread left blocked**. `main` must `pthread_join` all four threads and return 0.
- `N` = 200 total (100 + 100). You may expose it as a command-line option for testing, but it must default to 100 per producer.

---

8. Output and logging

Each thread writes to its **own** log file (so file writes never contend):

```
producer_odd_output.txt
producer_even_output.txt
consumer_odd_output.txt
consumer_even_output.txt
```

Log format — one line per action:

```
[seq=<n>] [thread=<name>] [action=
<before_insert|after_insert|before_remove|after_remove>] [value=
<v>] [size=<s>] [odd=<o>] [even=<e>]
```

Example:

```
[seq=23] [thread=producer_odd] [action=before_insert] [value=37]
[size=4] [odd=1] [even=3]
[seq=24] [thread=producer_odd] [action=after_insert]  [value=37]
[size=5] [odd=2] [even=3]
```

Here a single odd value (37) is inserted, so `size` rises 4 → 5 and `odd` rises 1 → 2 while `even` is unchanged. Nothing is removed — the `4` and `5` are item *counts*, not values, and

a producer only ever inserts.

- `value` is the integer inserted or removed.
- `size`, `odd`, `even` are the buffer totals **at that instant** — i.e., exactly the quantities your predicates test. Logging these (rather than the full contents) keeps the trace aligned with the conditions you are reasoning about.
- Producers log `before_insert` / `after_insert`; consumers log `before_remove` / `after_remove`.
- The `before` snapshot, the modification, and the `after` snapshot for one operation must be produced **atomically** with respect to other threads' operations (i.e., under the buffer lock), so another thread's operation never appears between a matched `before` / `after` pair.
- `seq` is a **single global, monotonically increasing** integer, incremented **inside the critical section** by whichever thread is about to write a line. Sorting all lines from all four files by `seq` therefore reconstructs a globally consistent order of operations. (You may capture the snapshot strings and `seq` under the lock and defer the actual file write until after unlocking, as long as `seq` is assigned under the lock.)

Wasted-wakeup reporting (required). Each thread counts how many times it resumes from a blocking wait only to find it still cannot proceed and must wait again. At termination, print each thread's count and the total to stdout (or a `summary.txt`). A design that blocks each waiter on the specific condition it needs keeps this near zero; a design that funnels everything through one shared condition and rouses all waiters produces many. (With well-structured semaphores this count is typically zero by construction; with condition variables it reflects how targeted your waking is.) This makes your blocking quality visible and gradable.

---

9. Deliverables

Submit to the Canvas Assignment 2 folder:

1. **Source code** (a POSIX-threads program in C or C++) and a **Makefile** that links pthreads (e.g. `-pthread`; suggested binary name `buffer_sim`).
2. **Four log files** from one run with a stated seed, plus the **wasted-wakeup summary**.
3. **README.md** with build/run instructions and the seed used.
4. **Design write-up** (`design.txt` or a section in the README) answering §10.
5. **A test script** that builds and runs with seed 42 and `N = 200`.
6. **analysis.pdf** *(only if applicable)* — if your program deadlocks or stalls, describe the root cause and the fixes you attempted.

---

10. Required design write-up (answer explicitly)

This is where the understanding shows; a working program that cannot answer these loses the design points.

1. **The predicates.** What are the distinct wait conditions in this problem, and what does each of the four threads wait on?
2. **Keeping the conditions distinct.** Why is a single "an item exists" condition insufficient here, and how does your solution let a consumer wait specifically for an odd (or even) item? If you used condition variables, give a concrete interleaving in which one shared "not empty" condition plus a single `signal` would leave an eligible consumer asleep forever.
3. **Your guarantee.** How does your design ensure no eligible thread is ever left blocked when its predicate becomes true?
4. **Targeting.** How does your design avoid repeatedly resuming a thread that still cannot proceed? Refer to your wasted-wakeup numbers, and explain what in your choice of primitives keeps them low.
5. **Termination.** Why does each consumer consume exactly 100 items, and why can no thread be left blocked at the end?
6. **Reproducibility.** How is each producer's value sequence reproducible despite concurrency, and why is the interleaving still nondeterministic?

---

11. Common pitfalls (self-check before submitting)

- *(condition variables)* One shared "not empty" condition + a single `pthread_cond_signal` → the wrong consumer wakes and the right one keeps sleeping: **lost wakeup / effective deadlock.**
- *(condition variables)* `if` instead of `while` around the wait → a thread proceeds when its predicate is actually false.
- *(semaphores)* Acquiring the mutex *before* the resource semaphore (e.g. `wait(mutex)` then `wait(space)`) → a full-buffer producer blocks while holding the mutex and freezes everyone. Wait on the resource semaphore first, then the mutex.
- Checking "is there an odd item?" and removing it in *separate* critical sections → the item can vanish in between. This is a **time-of-check to time-of-use (TOCTOU)** bug: the state you verified is no longer true by the time you act on it. Keep the check and the removal in one critical section.
- `rand()` from multiple threads → not thread-safe, not reproducible; use per-thread `rand_r()`.
- Terminating consumers on "empty + producers done" instead of the fixed count of 100 → can hang on the last items.
- Logging outside the lock so a `before` / `after` pair straddles another operation, or `seq` comes out of order.
- Inconsistent lock ordering or blocking while holding a lock on an unrelated resource → deadlock.

---

12. Testing

Provide a script that builds the program and runs it with seed 42 and `N = 200` (the total number of items produced and consumed — 100 per producer), producing the four logs and the wasted-wakeup summary. A useful automated check: merge all four logs, sort by `seq`, and verify that each line's `size` / `odd` / `even` follow from the previous operation, that `size` never exceeds 50, that every inserted value is later removed, and that each consumer performs exactly 100 removals.

---

13. Rubric (20 points)

Category	Points
Bounded buffer — insert and remove-by-type work; capacity cap (50) enforced; counts tracked correctly	2
Producers — correct parity and range; block when full; per-thread reproducible PRNG	3
Consumers — remove an item of the correct type; block on absence of *own type* (not on empty)	3
Mutual exclusion & data-race freedom — all shared state under the lock; check-and-act atomic; no busy-waiting	3
No lost wakeups — no eligible thread ever left blocked when its condition holds; correct whether implemented with semaphores or condition variables	3
Targeted blocking — each waiter blocks on the specific condition it needs, so it rarely resumes unable to proceed; low wasted-wakeup count, correctly reported	2
Termination — exact per-thread counts; clean shutdown; no thread left blocked; `main` joins all	2
Logging & output — correct format; atomic `before` →op→ `after`; global `seq` in the critical section; wasted-wakeup summary	1
Design write-up — the §10 questions answered correctly	1

Semaphores and condition variables are equally acceptable — use whichever fits your design. The **no-lost-wakeups** points are for correctness: any correct semaphore design earns them, as does a condition-variable design that wakes all waiters with `broadcast`. The **targeted-blocking** points reward keeping the three conditions separate so threads resume only when they can proceed — the distinction this assignment is really testing, and reachable with either primitive.

---

14. References

- **POSIX threads programming** — a concise pthreads tutorial covering thread creation, mutexes, and condition variables: https://web.cs.ucla.edu/classes/winter04/cs118/pthreads.html
- The POSIX manual pages are the authoritative API reference: `man pthreads`, plus individual calls such as `man pthread_mutex_lock`, `man pthread_cond_wait`, and `man sem_wait`.

---

15. Academic integrity

You may discuss concepts and your progress with classmates, but you must design and write your own code. **Do not use generative AI to produce the code.** Using AI to explain a concept to you (e.g., how condition variables or semaphores work) is fine; having it write, complete, or debug your submission is not. When in doubt, ask the instructor.